AI Pentest Agent

Reinforcement Learning that Learns to Find Web Vulnerabilities

Capstone Project Report

An agent that learns, by trial and error, to find security bugs in a practice web app — and beats random guessing six to one. Educational, sandbox-only.

RL · RLcapstone.ai

1. Executive Summary

This project builds a tiny practice website with three security bugs on purpose, then an AI agent that learns — by trial and error, the same idea behind game-playing AI — to find all three. Once trained it finds every bug in **4 actions**; a random agent needs about **25** and often misses one.

> **Educational and sandbox-only.** Everything runs against a purpose-built vulnerable app on localhost. The techniques are standard OWASP Top 10 teaching examples and must only be used on systems you own or are authorised to test.

2. Introduction & Motivation

Automated scanners are fast but blind: they follow fixed scripts, miss chained issues, and cannot adapt. Recent research shows reinforcement learning can learn efficient attack paths. This project explores that idea at a scale a student can fully understand — and, importantly, explain.

3. The Practice App & Vulnerabilities

The target is a ~120-line 'MiniBank' Flask app on localhost with three planted, classic flaws:

- SQL injection — the login query is built by string concatenation, so a crafted username logs in without a password.
- Reflected XSS — the search box echoes user input into the page unescaped.
- IDOR (broken access control) — changing the id in a profile URL reveals another user's private data.

4. Methodology

Penetration testing is modelled as a small game (a Markov Decision Process). The state records what has been discovered; the agent chooses among **12 actions** (reconnaissance, one working payload per bug, and several realistic dead ends). Every action is a **real HTTP request**, and a bug only counts when the real response proves it.

The agent is trained with **tabular Q-learning** — a 16×12 table of learned action-values that can be read and explained directly — rewarded +10 for each new bug, +2 for doing recon first, and -1 per step so shorter attacks score higher. It is not told which actions are good; it discovers them.

5. Results

Agent	Actions to find all 3	Success rate
Trained (Q-learning)	4	100%
Random	~25	50%

Over a few hundred practice runs the agent's average session reward climbs from deeply negative (flailing) to +28 (the clean four-step attack: recon, then one payload per bug). Learning turns flailing

into a repeatable strategy — about a six-fold efficiency gain over random guessing.

After finishing, the agent writes a structured penetration-test report of each finding (severity, OWASP category, impact, and remediation).

6. Limitations & Ethical Considerations

- A tiny sandbox: three known bugs and twelve actions — a teaching model, not a real scanner.
- Sandbox-only against a purpose-built target on localhost; standard OWASP teaching payloads only.
- Intended solely for educational and authorised, defensive security purposes.

7. Future Work

- More vulnerability types and a second practice app to test generalisation.
- Wire a real language model into the report-writing and payload seams.
- Compare against scripted tools and non-learning baselines.